

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA TP.HCM
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



LUẬN VĂN TỐT NGHIỆP

PHÁT TRIỂN NỀN TẢNG NHÚNG ĐA CHỨC NĂNG DỰA TRÊN YOCTO PROJECT CHO CÁC ỨNG DỤNG TỰ ĐỘNG HÓA TRONG CÔNG NGHIỆP

HK252

Giảng viên hướng dẫn: TS.Lê Trọng Nhân

STT	Họ & Tên	MSSV	Lớp	Đánh giá
1	Đỗ Tuấn Anh	2210050	MT22KT01	100%
2	Dương Minh Hiếu	2210978	MT22KT01	100%
3	Nguyễn Hạo Duy	2210512	MT22KT01	100%

TP.Hồ Chí Minh, tháng 5 năm 2025



Thông tin hội đồng chấm khóa luận tốt nghiệp

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số 476/QĐĐHCNTT ngày 30 tháng 02 năm 2024 của Hiệu trưởng Trường Đại học Bách Khoa - TP.HCM.

- | | |
|-------------|----------|
| 1. PGS. TS. | Chủ tịch |
| 2. TS. | Ủy viên |
| 3. ThS. | Thư ký |

Lời cảm ơn

Không ai đạt được điều gì đó to lớn mà không nhờ sự giúp đỡ của những người xung quanh, cho dù là trực tiếp hay gián tiếp đi nữa. Để hoàn thành được khóa luận này, tác giả may mắn nhận được nhiều sự giúp đỡ và hỗ trợ từ quý thầy, cô, anh chị, bạn bè và người thân. Tác giả xin dành những trang đầu tiên này để bày tỏ lòng tri ân của mình tới tất cả mọi người, những người đã đồng hành cùng nhóm trong khoảng thời gian vừa qua. Đầu tiên, tác giả xin gửi lời cảm ơn sâu sắc đến toàn thể các thầy cô của Trường Đại học Công nghệ Thông tin nói chung và các thầy cô khoa Mạng máy tính và Truyền thông nói riêng. Nhờ những kiến thức quý giá mà thầy cô đã truyền đạt, cũng như việc hỗ trợ tận tình trong suốt khoảng thời gian thực hiện, nhóm đã hoàn thành khóa luận và đạt được các kết quả đáng ghi nhận. Tác giả xin đặc biệt cảm ơn TS. Google là người đã truyền cảm hứng, tận tình hướng dẫn và hỗ trợ tận tình về kiến thức, tạo môi trường thuận lợi để nhóm có thể học hỏi, trao đổi với các bạn, các em trong nhóm nghiên cứu. Đây là những kiến thức, kinh nghiệm quý giá, không chỉ có tác dụng trong khóa luận tốt nghiệp này mà còn trong khoảng thời gian làm việc trong chặng đường tiếp theo. Cuối cùng, tôi xin bày tỏ lòng tri ân đến gia đình và người thân, những người đã luôn là những hậu phương vững chắc và luôn ủng hộ từng quyết định mà nhóm đưa ra. Mặc dù đã nỗ lực rất nhiều để luận văn được hoàn thiện nhất, song khó có thể tránh khỏi thiếu sót và hạn chế. Kính mong nhận được sự thông cảm và ý kiến đóng góp từ quý thầy cô và các bạn.

Mục Lục

1	Phần mở đầu	7
1.1	Đặt vấn đề	8
1.2	Lý do chọn đề tài	8
1.3	Mục tiêu nghiên cứu	8
1.4	Phạm vi và giới hạn nghiên cứu	8
1.5	Bố cục luận văn	8
2	Tổng quan dự án	9
3	Cơ sở lý thuyết	10
3.1	Giao tiếp phần cứng công nghiệp	11
3.2	Linux Device Driver	11
3.3	Bảo mật hệ điều hành Linux nhúng	11
3.3.1	Khái niệm bảo mật trong Linux nhúng	11
3.3.2	Giảm thiểu tấn công bề mặt (Attack Surface Reduction)	12
3.3.3	Bảo mật quyền truy cập hệ thống với SELinux	13
3.3.4	Bảo mật chống tấn công offline với dm-verity	14
3.4	Edge AI trong hệ thống nhúng	16
3.5	Giao diện người dùng trên thiết bị nhúng	16
3.6	Kết nối Cloud và IoT	16
3.6.1	Tổng quan về CoreIOT	16
3.6.2	Giao thức MQTT trong hệ thống nhúng	18
4	Thiết kế hệ thống	19
5	Triển khai và đánh giá	20
6	Sản phẩm demo tích hợp	21



7 Kết luận và hướng phát triển

22

Danh sách bảng

Danh sách hình

3.1	Tăng cường bảo mật cho hệ thống nhúng Yocto là một bài toán đa khía cạnh .	12
3.2	Bảo mật quyền truy cập hệ thống với SELinux trong Yocto	14
3.3	Bảo mật chống tấn công offline với dm-verity trong Yocto	15
3.4	Tổng quan về CoreIOT	16
3.5	Xây dựng quy tắc xử lý dữ liệu và sự kiện trong CoreIoT	17
3.6	Giao thức MQTT trong hệ thống nhúng	18

Chương 1

Phần mở đầu

- 1.1** Đặt vấn đề
- 1.2** Lý do chọn đề tài
- 1.3** Mục tiêu nghiên cứu
- 1.4** Phạm vi và giới hạn nghiên cứu
- 1.5** Bố cục luận văn

1.1 Đặt vấn đề

1.2 Lý do chọn đề tài

1.3 Mục tiêu nghiên cứu

1.4 Phạm vi và giới hạn nghiên cứu

1.5 Bố cục luận văn

Chương 2

Tổng quan dự án

2.1 Tổng quan về tự động hóa công nghiệp

2.2 Tổng quan về hệ điều hành nhúng

2.3 Tổng quan về Yocto Project

2.4 Các công trình liên quan

Chương 3

Cơ sở lý thuyết

- 3.1** Giao tiếp phần cứng công nghiệp
- 3.2** Linux Device Driver
- 3.3** Bảo mật hệ điều hành Linux nhúng
- 3.4** Edge AI trong hệ thống nhúng
- 3.5** Giao diện người dùng trên thiết bị nhúng
- 3.6** Kết nối Cloud và IoT

3.1 Giao tiếp phần cứng công nghiệp

3.2 Linux Device Driver

3.3 Bảo mật hệ điều hành Linux nhúng

3.3.1 Khái niệm bảo mật trong Linux nhúng

Tăng cường bảo mật (security hardening) là quá trình áp dụng các biện pháp cụ thể nhằm mục tiêu nâng cao sự an toàn cho toàn hệ thống một cách toàn diện. Quá trình không chỉ dừng lại ở việc thiết lập các biện pháp nhằm giảm thiểu tấn công bề mặt (attack surface), mà còn bao gồm việc bảo vệ hệ thống ở nhiều lớp khác nhau, từ kernel, hệ điều hành, dịch vụ cho đến ứng dụng và dữ liệu.

Có nhiều lý do khiến việc tăng cường bảo mật ở các hệ thống nhúng Linux trở thành ưu tiên quan trọng đối với các nhà sản xuất thiết bị, trong đó có ba yếu tố chính cần được đặc biệt quan tâm.

Thứ nhất, khi số lượng các thiết bị nhúng được kết nối Internet ngày càng tăng, các thiết bị này trở thành sự lựa chọn tiềm năng, là mục tiêu cho các cuộc tấn công mạng. Việc kết nối Internet khiến thiết bị dễ dàng bị phát hiện và quét từ bên ngoài. Nếu thiết bị tồn tại lỗ hổng bảo mật, kẻ tấn công có thể khai thác để xâm nhập vào cả hệ thống.

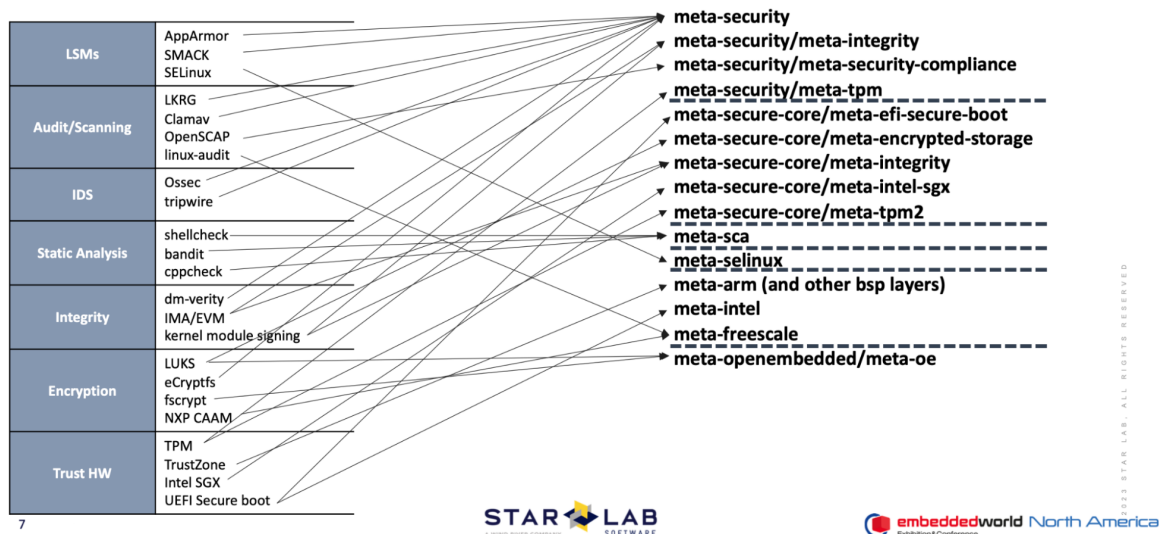
Thứ hai, nếu không có biện pháp bảo mật, các thiết bị nhúng rất dễ để lộ thông tin. Như Scott Murray của Konsulko Group đã giải thích tại Yocto Project Virtual DevDay năm 2020, việc quét tất cả các thiết bị IPv4 bằng công cụ tìm kiếm như shodan.io để xác định thiết bị nào đang hoạt động là điều rất dễ dàng [1]. Bên cạnh đó, việc cấu hình không đúng, chẳng hạn như mở cổng không cần thiết hoặc sử dụng các cơ chế tự động như UPnP, có thể khiến các dịch vụ trên thiết bị vô tình bị công khai ra bên ngoài. Những yếu tố này làm cho thiết bị không chỉ là mục tiêu tấn công, mà còn dễ dàng bị tìm thấy và khai thác, từ đó làm gia tăng đáng kể nguy cơ mất an toàn cho hệ thống.

Thứ ba, các thiết bị nhúng Linux có thể bị chiếm quyền điều khiển và sử dụng như một công cụ để tấn công các hệ thống khác, chẳng hạn như tham gia vào mạng botnet hoặc thực hiện tấn công DDoS. Trong trường hợp này, kẻ tấn công không nhắm vào dữ liệu của thiết bị mà khai thác khả năng kết nối và tài nguyên xử lý của nó. Do đó, ngay cả các thiết bị đơn giản cũng cần được bảo mật để tránh bị lợi dụng cho các mục đích tấn công.

Chính vì những rủi ro nêu trên, việc tăng cường bảo mật trở thành một trong những ưu tiên hàng đầu đối với các nhà sản xuất thiết bị, đồng thời là yếu tố xuyên suốt trong quá trình phát triển phần mềm nhúng và các hệ thống kết nối. Đối với các hệ thống sử dụng Linux, Yocto được xem là một công cụ quan trọng, cho phép các developer xây dựng và kiểm soát hệ thống

một cách linh hoạt, từ đó tích hợp các cơ chế bảo mật phù hợp ngay từ giai đoạn thiết kế.

Ngày nay, nhiều hệ thống nhúng Linux được xây dựng bằng Yocto Project đang được áp dụng rộng rãi trong các lĩnh vực trọng yếu nhưng có độ bảo mật chưa cao. Khác với các bản phân phối Linux phổ biến như Debian hay Ubuntu, Yocto không cung cấp một hệ điều hành hoàn chỉnh mà cung cấp các công nghệ tùy biến để các developer xây dựng một hệ điều hành theo nhu cầu. Điều này góp phần xây dựng một hệ điều hành có sự linh hoạt cao nhưng đồng thời đi cùng các rủi ro về bảo mật. Đối với các ứng dụng như hệ thống tự động trong công nghiệp, lĩnh vực sức khỏe hay xe tự hành thì tầm quan trọng của bảo mật lại càng trở nên cấp thiết hơn.



Hình 3.1: Tăng cường bảo mật cho hệ thống nhúng Yocto là một bài toán đa khía cạnh

Nguồn: Star Lab Software

3.3.2 Giảm thiểu tấn công bề mặt (Attack Surface Reduction)

Giảm thiểu tấn công bề mặt (attack surface reduction) là một trong những nguyên tắc quan trọng trong bảo mật hệ thống nhúng Linux. Mục tiêu của phương pháp này là hạn chế tối đa số lượng thành phần, dịch vụ, giao diện và cấu hình có thể bị khai thác bởi kẻ tấn công. Trong hệ thống nhúng, bề mặt tấn công thường bao gồm các dịch vụ mạng không cần thiết, các tính năng hệ điều hành dư thừa, các thành phần phần mềm không sử dụng và các thiết lập mặc định có thể làm giảm mức độ an toàn của hệ thống [1].

Đối với các hệ thống xây dựng bằng Yocto, việc giảm thiểu tấn công bề mặt có ý nghĩa đặc biệt quan trọng vì Yocto cho phép tùy biến sâu đối với image hệ điều hành. Chính sự linh hoạt này giúp hệ thống chỉ bao gồm những thành phần thực sự cần thiết, nhưng đồng thời cũng đòi hỏi nhà phát triển phải kiểm soát chặt chẽ các thành phần được đưa vào image. Nếu không được cấu hình cẩn thận, hệ thống có thể vô tình chứa nhiều gói phần mềm, dịch vụ hoặc tính năng mặc định không cần thiết, từ đó làm gia tăng nguy cơ xuất hiện lỗ hổng bảo mật.

Một số nguyên tắc phổ biến trong giảm thiểu tấn công bề mặt bao gồm: loại bỏ các dịch vụ không sử dụng, giới hạn quyền truy cập mặc định, tối giản hóa image hệ điều hành, chỉ giữ lại các thành phần cần thiết cho mục đích vận hành và hạn chế các khả năng truy cập ở mức thấp của người dùng đối với hệ thống. Ngoài ra, việc tinh chỉnh cấu hình kernel cũng đóng vai trò quan trọng vì kernel là lớp điều khiển trực tiếp phần cứng và có ảnh hưởng lớn đến mức độ an toàn tổng thể của hệ thống [2,3].

Từ góc nhìn lý thuyết, giảm thiểu tấn công bề mặt không chỉ giúp hạn chế khả năng khai thác trực tiếp mà còn góp phần nâng cao độ ổn định, độ tin cậy và khả năng bảo trì của hệ thống nhúng trong quá trình vận hành lâu dài.

3.3.3 Bảo mật quyền truy cập hệ thống với SELinux

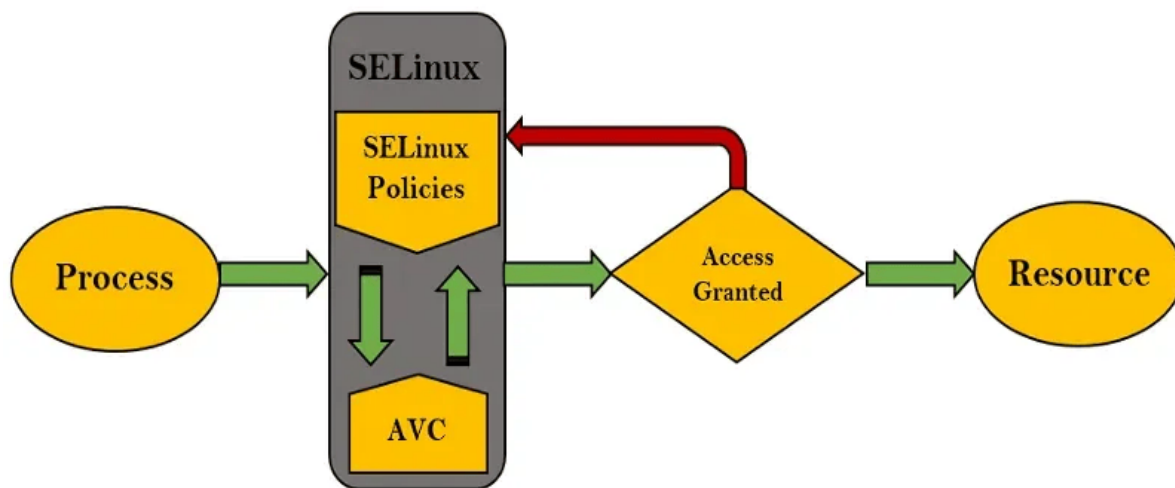
Security-Enhanced Linux (SELinux) là một kiến trúc bảo mật dành cho các hệ thống Linux, cho phép kiểm soát chặt chẽ việc truy cập tài nguyên trong hệ điều hành. SELinux được công bố cho cộng đồng mã nguồn mở vào năm 2000 và chính thức được tích hợp vào nhân Linux từ năm 2003 [4]. Khác với cơ chế phân quyền truyền thống dựa trên mô hình kiểm soát truy cập tùy ý (Discretionary Access Control - DAC), SELinux áp dụng mô hình kiểm soát truy cập bắt buộc (Mandatory Access Control - MAC) nhằm thực thi các chính sách bảo mật một cách tập trung và nghiêm ngặt. Trong mô hình này, mọi hành động truy cập đều phải tuân theo các quy tắc được định nghĩa trước; nếu không được cho phép rõ ràng, truy cập sẽ bị từ chối theo mặc định. Điều này giúp hạn chế hiệu quả các hành vi truy cập trái phép và nâng cao mức độ an toàn tổng thể của hệ thống [5].

SELinux hoạt động bằng cách gán ngữ cảnh bảo mật (security context) cho cả chủ thể và đối tượng trong hệ thống. Chủ thể thường là các tiến trình hoặc dịch vụ hệ thống, trong khi đối tượng có thể là tệp, thư mục, socket hoặc các tài nguyên hệ thống khác. Mỗi ngữ cảnh bảo mật bao gồm các thông tin như user, role, type và mức độ nhạy cảm, từ đó xác định quyền truy cập của từng thực thể. Khi một tiến trình yêu cầu truy cập tài nguyên, cơ chế thực thi chính sách của SELinux sẽ so sánh ngữ cảnh bảo mật của chủ thể và đối tượng với các quy tắc đã được định nghĩa trong policy. Nếu hành động phù hợp với chính sách, quyền truy cập sẽ được cấp; ngược lại, hành động sẽ bị từ chối và có thể được ghi lại trong nhật ký hệ thống [5].

Một trong những đặc điểm quan trọng của SELinux là nguyên tắc đặc quyền tối thiểu (principle of least privilege), theo đó mỗi tiến trình chỉ được cấp các quyền cần thiết để thực hiện chức năng của mình. Cơ chế này giúp hạn chế khả năng lan rộng của các cuộc tấn công trong trường hợp một tiến trình hoặc dịch vụ bị khai thác. Ngoài ra, SELinux còn hỗ trợ cơ chế ghi nhật ký và giám sát, giúp quản trị viên dễ dàng phát hiện và phân tích các hành vi bất thường trong hệ thống.

Trong các hệ thống xây dựng bằng Yocto, SELinux không được kích hoạt mặc định mà thường được tích hợp thông qua các layer bổ sung như meta-selinux, meta-security

hoặc meta-smack. Các layer này cung cấp các thành phần cần thiết để hệ thống hỗ trợ mô hình MAC, từ đó giúp tăng cường khả năng bảo vệ cho các hệ thống Linux nhúng.



Hình 3.2: Bảo mật quyền truy cập hệ thống với SELinux trong Yocto

Nguồn: <https://www.geeksforgeeks.org/linux-unix/what-is-selinux/>

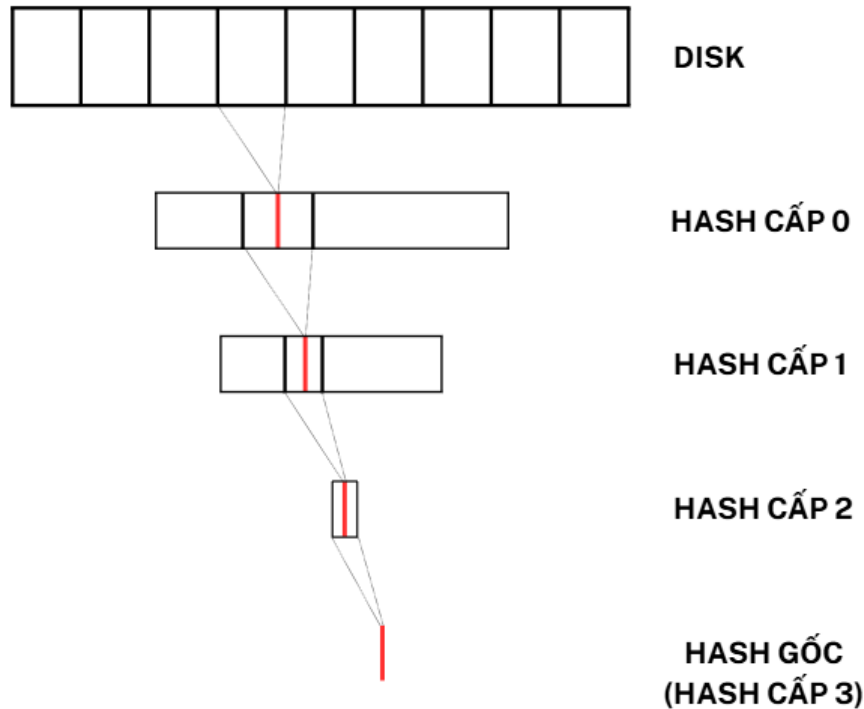
3.3.4 Bảo mật chống tấn công offline với dm-verity

Hầu hết các cơ chế bảo mật trong Linux tập trung vào việc bảo vệ hệ thống trong quá trình vận hành. Tuy nhiên, vẫn tồn tại các hình thức tấn công xảy ra khi hệ thống không hoạt động, chẳng hạn như việc tháo thiết bị lưu trữ để thay đổi dữ liệu trực tiếp rồi lắp lại vào hệ thống. Để phát hiện và hạn chế các cuộc tấn công dạng này, Linux cung cấp nhiều cơ chế kiểm tra tính toàn vẹn của hệ thống tập tin, trong đó dm-verity là một giải pháp được sử dụng phổ biến trong các hệ thống nhúng nhờ khả năng bảo vệ dữ liệu mạnh, chống lại các tấn công offline và không yêu cầu sử dụng khóa bí mật [6].

dm-verity (device-mapper verity) là một cơ chế xác minh tính toàn vẹn dữ liệu ở mức block device trong Linux kernel. Cơ chế này hoạt động dựa trên hệ thống device-mapper và thường được sử dụng để bảo vệ root filesystem ở chế độ chỉ đọc (read-only root filesystem). Khi được kích hoạt, mọi khối dữ liệu được đọc từ thiết bị lưu trữ đều sẽ được xác minh trước khi chuyển cho hệ điều hành sử dụng. Nếu phát hiện dữ liệu bị thay đổi hoặc không khớp với giá trị băm đã lưu trước đó, dm-verity sẽ báo lỗi hoặc từ chối mount filesystem nhằm ngăn chặn việc sử dụng dữ liệu không tin cậy.

Để thực hiện việc xác minh dữ liệu một cách hiệu quả, dm-verity sử dụng cấu trúc cây băm Merkle (Merkle tree). Trong cấu trúc này, mỗi khối dữ liệu sẽ được tính toán giá trị hash riêng biệt. Sau đó, các hash này tiếp tục được ghép cặp và băm lại để tạo thành các lớp hash

cấp cao hơn. Quá trình này được lặp lại cho đến khi thu được một giá trị hash duy nhất ở đỉnh cây, gọi là root hash. Root hash đại diện cho toàn bộ dữ liệu của filesystem và đóng vai trò là giá trị tin cậy để xác minh tính toàn vẹn của toàn hệ thống.



Hình 3.3: Bảo mật chống tấn công offline với dm-verity trong Yocto

Một ưu điểm quan trọng của Merkle tree là hệ thống không cần xác minh toàn bộ filesystem cùng một lúc. Thay vào đó, dm-verity chỉ cần kiểm tra các hash liên quan đến block dữ liệu đang được truy cập. Điều này giúp giảm đáng kể chi phí tính toán và thời gian khởi động hệ thống so với phương pháp sử dụng một hash duy nhất cho toàn bộ filesystem. Ngoài ra, nếu một block dữ liệu bị thay đổi, chỉ nhánh hash liên quan mới bị ảnh hưởng, trong khi các phần còn lại của hệ thống vẫn có thể được xác minh độc lập.

Khi dữ liệu được truy cập lần đầu tiên, kernel sẽ đọc block dữ liệu từ thiết bị lưu trữ và tiến hành xác minh thông qua cây băm. Hệ thống sẽ lần lượt truy xuất các hash từ các cấp thấp hơn cho đến root hash đã được tin cậy trước đó. Nếu toàn bộ quá trình xác minh thành công, dữ liệu sẽ được chấp nhận; ngược lại, hệ thống sẽ phát hiện dữ liệu đã bị thay đổi hoặc hỏng. Trong các lần truy cập tiếp theo, dữ liệu và các hash liên quan có thể đã được lưu trong bộ nhớ đệm (cache), cho phép hệ điều hành tái sử dụng các kết quả xác minh trước đó mà không cần đọc lại toàn bộ cây băm. Điều này giúp cải thiện đáng kể hiệu năng truy cập dữ liệu trong quá trình vận hành.

Nhờ khả năng xác minh tính toàn vẹn ở mức block device, dm-verity đặc biệt phù hợp với các hệ thống nhúng yêu cầu tính ổn định và độ tin cậy cao như thiết bị IoT, hệ thống công nghiệp hoặc các nền tảng Linux tối giản. Cơ chế này giúp phát hiện các thay đổi trái phép đối

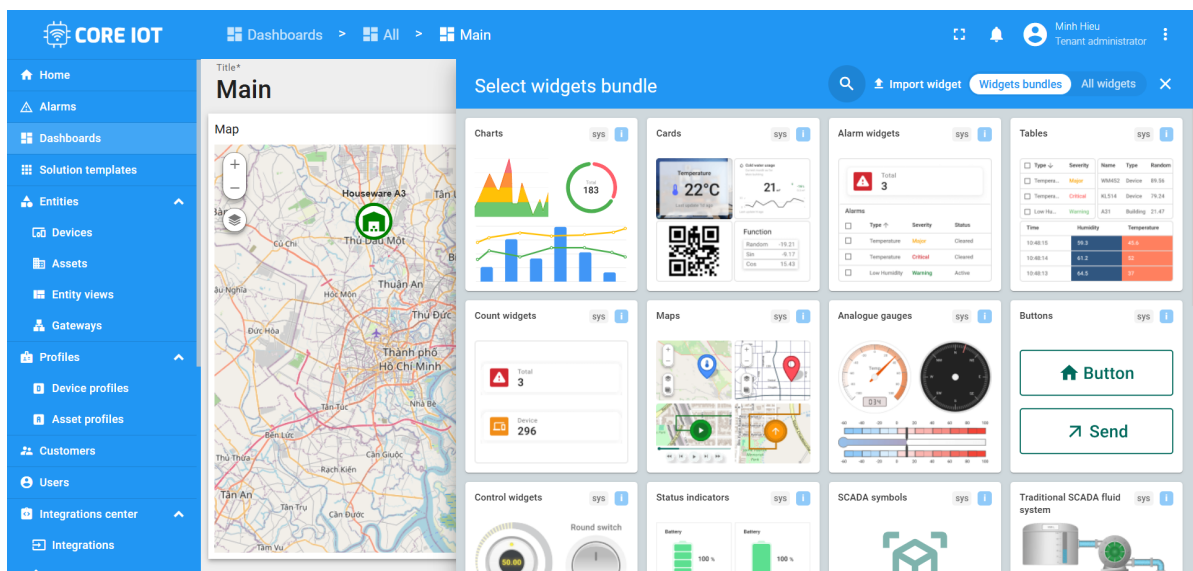
với root filesystem, đồng thời tăng cường khả năng chống lại các cuộc tấn công ngoại tuyến nhằm sửa đổi dữ liệu hệ thống.

3.4 Edge AI trong hệ thống nhúng

3.5 Giao diện người dùng trên thiết bị nhúng

3.6 Kết nối Cloud và IoT

3.6.1 Tổng quan về CoreIoT



Hình 3.4: Tổng quan về CoreIoT

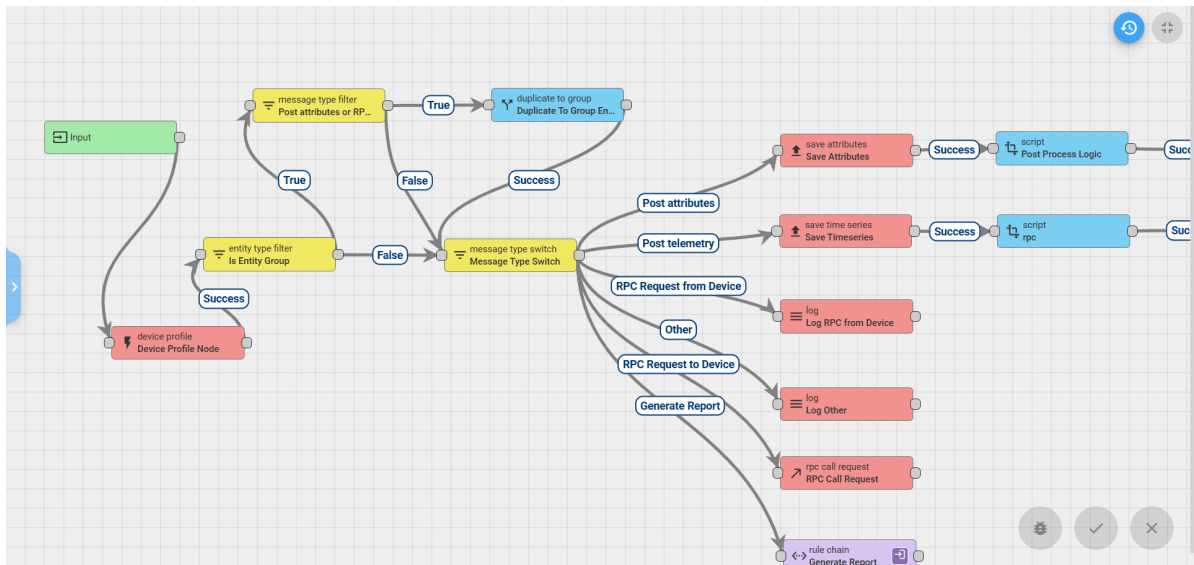
CoreIoT là một nền tảng Internet of Things (IoT) được phát triển và xây dựng dựa trên định hướng tương tự các nền tảng IoT mã nguồn mở phổ biến như ThingsBoard, nhằm phục vụ nhu cầu giảng dạy, nghiên cứu và triển khai các bài toán IoT trong môi trường học thuật và đào tạo kỹ thuật. Nền tảng được tùy biến và phát triển lại để phù hợp với điều kiện sử dụng, ngôn ngữ và đối tượng người dùng là sinh viên, giảng viên và các nhóm nghiên cứu tại nhà trường.

Về mặt kiến trúc tổng thể, CoreIoT được thiết kế theo mô hình nền tảng trung tâm, cho phép thu thập, xử lý, lưu trữ và hiển thị dữ liệu từ các thiết bị IoT phân tán. Hệ thống đóng vai trò trung gian giữa các thiết bị đầu cuối và người dùng, hỗ trợ quản lý thiết bị, giám sát dữ liệu theo thời gian thực và xây dựng các ứng dụng IoT hoàn chỉnh. Cách tiếp cận này giúp người học dễ dàng tiếp cận các khái niệm cốt lõi của IoT mà không cần triển khai toàn bộ hạ tầng từ đầu.

CoreIoT hỗ trợ kết nối với các thiết bị IoT thông qua các giao thức truyền thông phổ biến, đặc biệt phù hợp với các thiết bị nhúng và hệ thống công nghiệp. Dữ liệu từ cảm biến

sau khi được thu thập tại thiết bị biên sẽ được truyền về nền tảng để xử lý và lưu trữ tập trung. Thông qua đó, CoreIoT cho phép theo dõi trạng thái thiết bị, phân tích dữ liệu và hiển thị thông tin trực quan trên dashboard, hỗ trợ người dùng giám sát hệ thống một cách hiệu quả.

Một điểm nổi bật của CoreIoT là giao diện người dùng được thiết kế theo hướng trực quan, thân thiện. Các chức năng quản lý thiết bị, cấu hình dashboard và theo dõi dữ liệu được đơn giản hóa, giúp sinh viên và người mới tiếp cận IoT có thể nhanh chóng làm quen và khai thác nền tảng. Điều này góp phần giảm rào cản kỹ thuật trong quá trình học tập và thực hành, đồng thời nâng cao hiệu quả đào tạo.

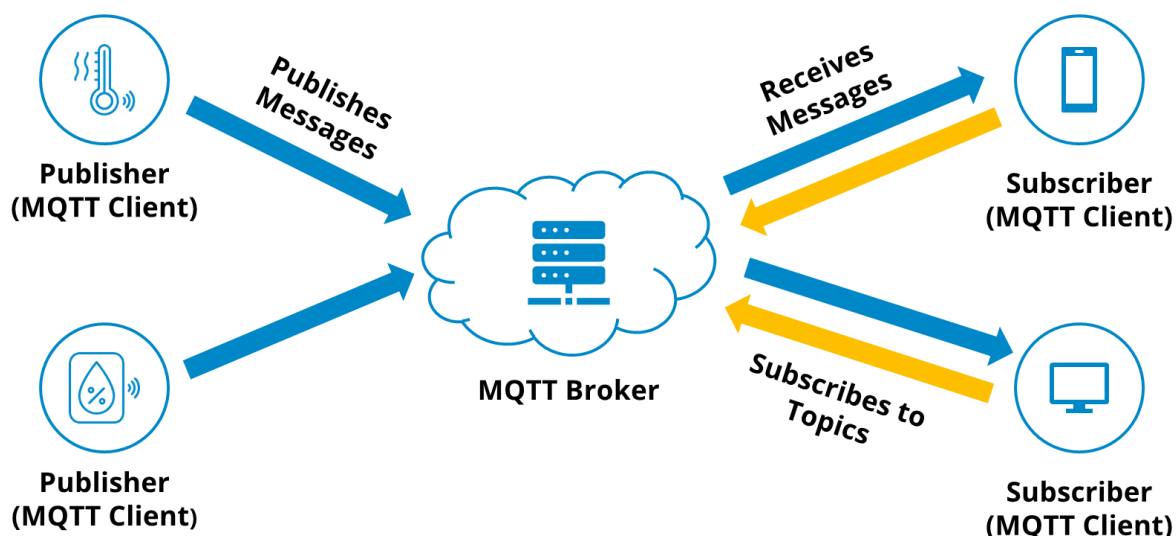


Hình 3.5: Xây dựng quy tắc xử lý dữ liệu và sự kiện trong CoreIoT

Bên cạnh chức năng giám sát, CoreIoT còn hỗ trợ xây dựng các quy tắc xử lý dữ liệu và sự kiện thông qua cơ chế xử lý logic ở tầng nền tảng. Các quy tắc này cho phép hệ thống phản ứng tự động trước các điều kiện cụ thể, chẳng hạn như phát cảnh báo khi giá trị cảm biến vượt ngưỡng hoặc ghi nhận sự kiện phục vụ phân tích sau này. Cách tiếp cận này giúp người học hiểu rõ hơn về mô hình xử lý dữ liệu và tự động hóa trong các hệ thống IoT hiện đại.

Với định hướng phục vụ đào tạo và nghiên cứu, CoreIoT không chỉ đóng vai trò là một công cụ giám sát mà còn là môi trường thực hành giúp sinh viên tiếp cận các khái niệm quan trọng như quản lý thiết bị, truyền thông IoT, xử lý dữ liệu thời gian thực và tích hợp hệ thống. Nền tảng tạo điều kiện thuận lợi cho việc triển khai các đề tài học tập, đồ án và nghiên cứu ứng dụng, đồng thời có khả năng mở rộng để đáp ứng các bài toán IoT phức tạp hơn trong tương lai.

3.6.2 Giao thức MQTT trong hệ thống nhúng



Hình 3.6: Giao thức MQTT trong hệ thống nhúng

Nguồn: <https://www.manubus.com/what-is-mqtt/>

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền tin nhẹ, được thiết kế theo mô hình publish–subscribe, đặc biệt phù hợp cho các hệ thống IoT có tài nguyên hạn chế và yêu cầu truyền dữ liệu với độ trễ thấp. Trong mô hình này, các thiết bị không giao tiếp trực tiếp với nhau mà thông qua một thành phần trung gian gọi là broker. Thiết bị gửi dữ liệu sẽ đóng vai trò publisher, trong khi thiết bị nhận dữ liệu là subscriber, và broker chịu trách nhiệm phân phối thông điệp dựa trên các chủ đề (topic).

Một đặc điểm nổi bật của MQTT là kích thước gói tin nhỏ, giúp giảm băng thông và tiết kiệm năng lượng, rất phù hợp với các thiết bị nhúng và mạng truyền thông không ổn định. MQTT hỗ trợ nhiều mức chất lượng dịch vụ (Quality of Service – QoS), cho phép cân bằng giữa độ tin cậy và hiệu năng truyền thông. Bên cạnh đó, giao thức còn hỗ trợ cơ chế giữ kết nối (keep-alive), thông báo trạng thái thiết bị và khả năng mở rộng linh hoạt trong các hệ thống IoT quy mô lớn. Nhờ các đặc tính trên, MQTT thường được sử dụng trong các ứng dụng giám sát thời gian thực, thu thập dữ liệu cảm biến, hệ thống điều khiển từ xa và các nền tảng IoT trung tâm như CoreIoT hoặc các nền tảng tương đương.

Chương 4

Thiết kế hệ thống

Chương này tập trung vào thiết kế 3 thành phần kỹ thuật cốt lõi của nền tảng: Driver phần cứng, Bảo mật hệ thống và Công cụ hỗ trợ phát triển.

4.1 Kiến trúc tổng thể

4.2 Thiết kế Device Driver

4.3 Thiết kế Bảo mật Hệ thống

4.4 Thiết kế Configuration Tool

Chương 5

Triển khai và đánh giá

Chương này trình bày quá trình cài đặt, tích hợp và kết quả kiểm thử từng thành phần kỹ thuật của nền tảng.

- 5.1** Môi trường phát triển
- 5.2** Hiện thực Device Driver
- 5.3** Hiện thực Bảo mật Hệ thống
- 5.4** Hiện thực Configuration Tool

Chương 6

Sản phẩm demo tích hợp

Chương này là minh chứng thực nghiệm cho toàn bộ nền tảng: một hệ thống tự động hóa công nghiệp hoàn chỉnh được xây dựng trên nền tảng đã phát triển ở các chương trước.

- 6.1** Tổng quan sản phẩm demo
- 6.2** Kiến trúc hệ thống demo
- 6.3** Thiết kế và triển khai từng module
- 6.4** Tích hợp và kiểm thử sản phẩm demo

Chương 7

Kết luận và hướng phát triển

7.1 Kết luận

7.2 Hướng phát triển

Tài liệu tham khảo

- [1] S. Murray, “Security hardening with openembedded / yocto,” https://wiki.yoctoproject.org/wiki/images/archive/0/0d/20200702141904!DD5_Security_Hardening_NA20.pdf, 2020, accessed: 2026-05-09.
- [2] Witekio, “Yocto kernel development and security hardening,” <https://witekio.com/blog/yocto-kernel-development-security-hardening/>, 2024, accessed: 2026-05-09.
- [3] Z. Zhang *et al.*, “Kasr: A reliable and practical approach to attack surface reduction of commodity os kernels,” *arXiv preprint arXiv:1802.07062*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.07062>
- [4] Red Hat, “What is selinux?” <https://www.redhat.com/en/topics/linux/what-is-selinux>, Aug. 2019, accessed: 2026-03-26.
- [5] A. Muhammad, “What is selinux? how does it enhance linux security?” <https://www.hostinger.com/tutorials/what-is-selinux>, Jun. 2025, accessed: 2026-03-26.
- [6] Yocto Project, “[meta-security,1/2] dm-verity: add basic non-arch/non-bsp yocto specific settings,” <https://patchwork.yoctoproject.org/project/yocto/patch/20230310181117.3344359-2-paul.gortmaker@windriver.com/>, 2023, patchwork discussion, Accessed: 2026-03-26.